Biyomedikal İşaret ve Görüntü İşleme Ödev 3: DTFT Spektrum

Muhammed Kayra Bulut - 25501805

Ekim 2025

1 Giriş

Bu çalışmada, farklı ayrık zaman sinyallerinin Ayrık Zamanlı Fourier Dönüşümü (DTFT) spektrum analizleri gerçekleştirilmiştir. DTFT, biyomedikal sinyal işlemede sinyallerin frekans bileşenlerini analiz etmek için kullanılan temel bir araçtır. Zaman domenindeki sinyallerin frekans domenine dönüştürülmesi, sinyallerin karakteristik özelliklerinin daha iyi anlaşılmasını sağlar.

1.1 DTFT Sonuçlarının Analizi

1.1.1 Şekil 1

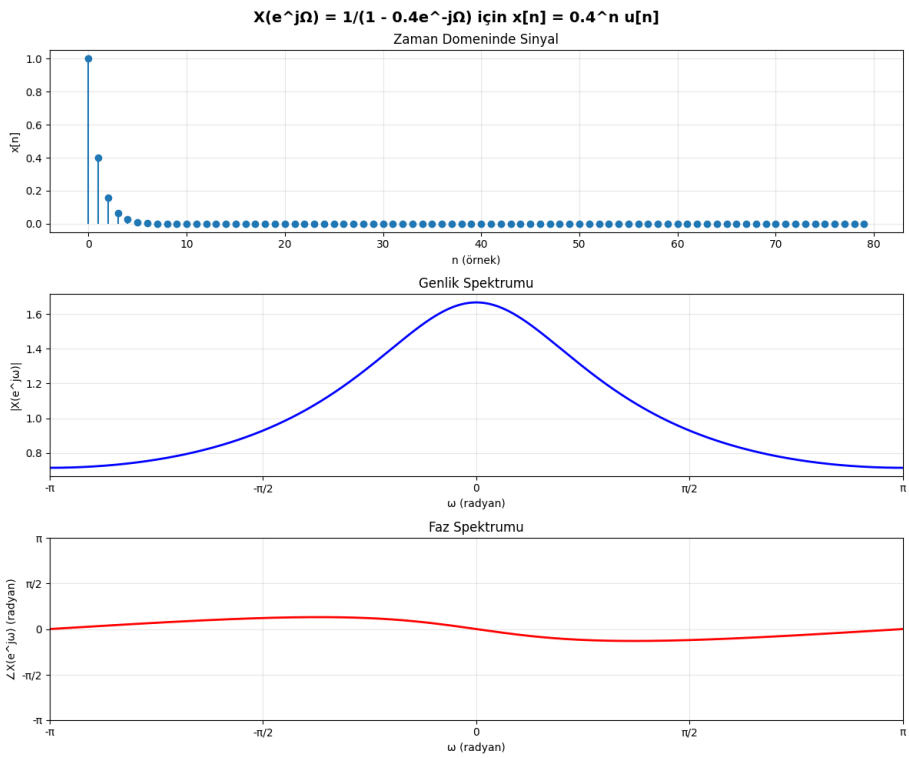
Bu şekilde 0.4 parametreli yavaş sönümlü bir üstel sinyalin DTFT analizi gösterilmektedir. Zaman domeninde sinyal hızla sıfıra yaklaşmakta, genlik spektrumunda düşük frekanslarda maksimum değer gözlenmektedir. Faz spektrumu nispeten düz bir karakteristik göstermekte, bu da sinyalin fazının frekansla yavaş değiştiğini göstermektedir. Düşük alfa değeri, sistemin geniş bant genişliğine sahip olduğunu ve daha hızlı geçici yanıt verdiğini işaret eder.

1.1.2 Şekil 2

Bu şekilde 0.8 parametreli daha yavaş sönümlü bir üstel sinyalin DTFT analizi sunulmaktadır. Zaman domeninde sinyal daha yavaş azalmakta, bu da genlik spektrumunda 0 civarında çok daha keskin bir tepe oluşmasına neden olmaktadır. Maksimum genlik değeri yaklaşık 5'e ulaşmaktadır. Faz spektrumunda ise 0 civarında ani bir faz değişimi gözlenmekte, bu da kutbun birim çembere daha yakın olmasından kaynaklanmaktadır. Yüksek alfa değeri, dar bant genişliği ve daha uzun geçici yanıt süresini gösterir.

1.1.3 Şekil 3

Bu şekilde iki farklı üstel sinyalin toplamından oluşan bileşik bir sinyalin DTFT analizi gösterilmektedir. İlk bileşen 4 örnek gecikmeli ve 0.2 parametreli, ikinci

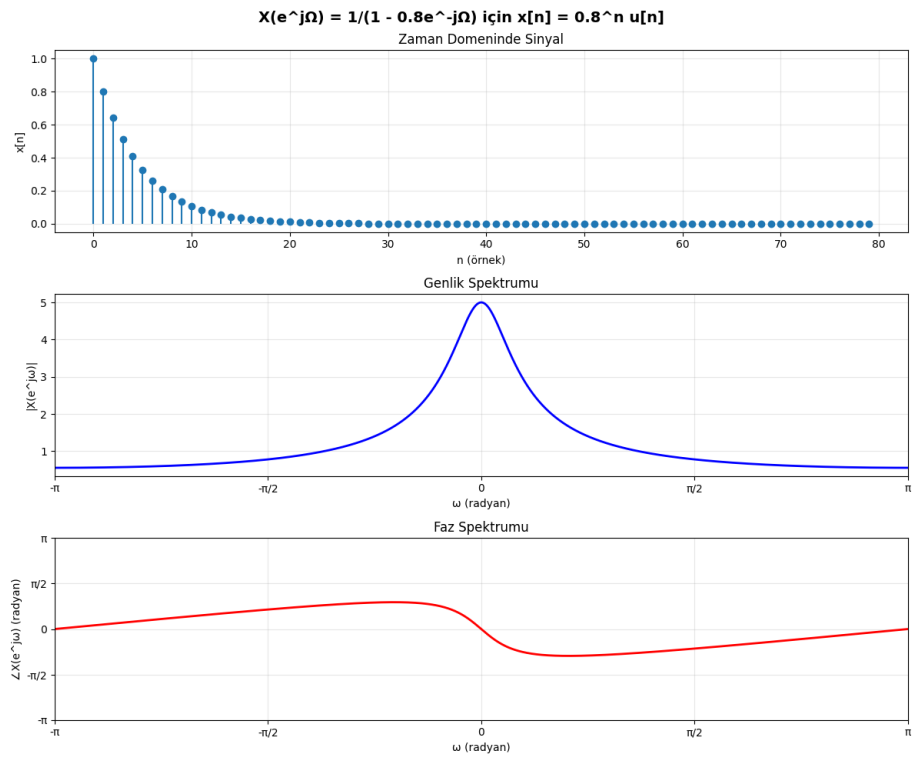


Şekil 1:

bileşen ise 2.5 kat ölçeklenmiş ve 0.4 parametreli bir sinyaldir. Zaman domeninde başlangıç değeri 2.5 civarındadır. Genlik spektrumu, her iki bileşenin etkisini yansıtarak 0'da maksimum değer göstermekte ve yaklaşık 4 genliğe ulaşmaktadır. Faz spektrumu, gecikme ve ölçekleme etkilerini birleştirerek karmaşık bir yapı sergilemektedir.

1.1.4 Şekil 4

Bu şekilde sonlu dürtü yanıtlı (FIR) bir filtrenin DTFT analizi sunulmaktadır. Zaman domeninde dürtü yanıtı ilk birkaç örnekte değer almakta ve hızla sönümlenmektedir. Genlik spektrumu yüksek frekanslarda daha yüksek değerler göstermekte, bu da filtrenin yüksek geçiren karakteristik özelliğe sahip olduğunu işaret etmektedir. Minimum genlik değeri orta frekanslarda yaklaşık 0.35 civarındadır. Faz spektrumu nispeten düzgün bir değişim göstermektedir. Bu tür filtreler, biyomedikal sinyallerde gürültü filtreleme ve özellik çıkarma uygulamalarında kullanılır.



Şekil 2:

2 Kod

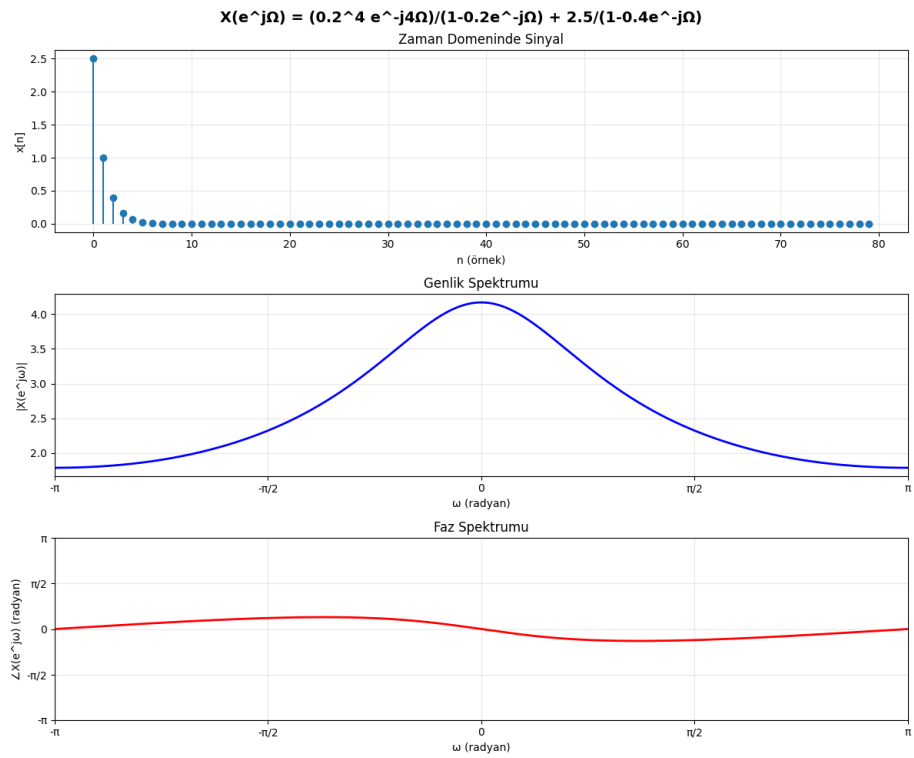
Kod aşağıda verilmiştir:

```
import os
import numpy as np
import matplotlib.pyplot as plt
from dtft_analyzer import DTFTAnalyzer

def main():
    # DTFT Analiz nesnesi
    analyzer = DTFTAnalyzer(num_points=2048)

    N = 80
    n = np.arange(0, N)

    a_values = [0.4, 0.8]
    signals = []
    for a in a_values:
        x = (a ** n) * (n >= 0)
```



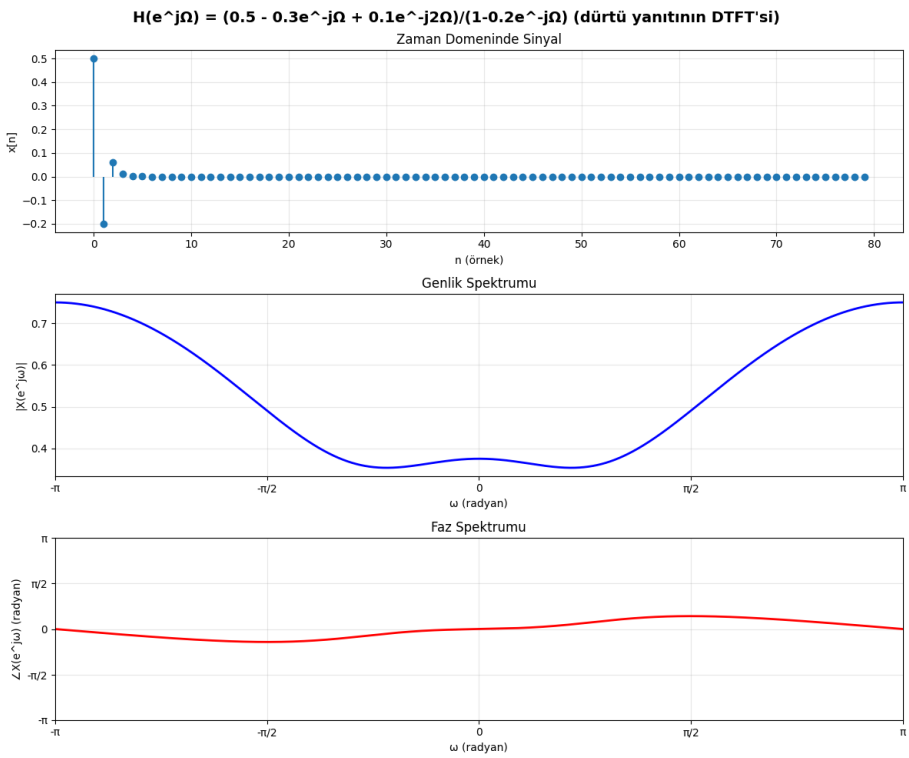
Şekil 3:

```
    title = f"1"
    signals.append((x, n, title))

x_comp = (0.2 ** n) * (n >= 4) + 2.5 * (0.4 ** n) * (n
    >= 0)
title_comp = "X(e^j$\omega$) = (0.2^4 e^-j4$\omega$)
    /(1-0.2e^-j$\omega$) + 2.5/(1-0.4e^-j$\omega$)"
signals.append((x_comp, n, title_comp))

h = (
    0.5 * (0.2 ** n) * (n >= 0)
    - 0.3 * (0.2 ** (n - 1)) * (n >= 1)
    + 0.1 * (0.2 ** (n - 2)) * (n >= 2)
)
title_h = "2"
signals.append((h, n, title_h))

for signal, nn, title in signals:
    fig = analyzer.plot_dtft_spectrum(signal, nn, title)
    safe_title = title.replace(os.sep, '_')
```



Şekil 4:

```
        plt.savefig(f"dtft_spektrum_analizi_{safe_title}.png
            ", bbox_inches='tight')
        plt.close(fig)

if __name__ == "__main__":
    main()
```

Listing 1: main.py

```
import numpy as np
import matplotlib.pyplot as plt

class DTFTAnalyzer:

    def __init__(self, num_points=1024):
        self.num_points = num_points
        self.omega = np.linspace(-np.pi, np.pi, num_points)

    def compute_dtft(self, signal, n):
        X = np.zeros(len(self.omega), dtype=complex)
```

```
        for i, w in enumerate(self.omega):
            X[i] = np.sum(signal * np.exp(-1j * w * n))

        return X

    def get_magnitude_phase(self, X):
        magnitude = np.abs(X)
        phase = np.angle(X)

        return magnitude, phase

    def plot_dtft_spectrum(self, signal, n, title="DTFT
        Analizi"):
        # DTFT hesapla
        X = self.compute_dtft(signal, n)
        magnitude, phase = self.get_magnitude_phase(X)


        fig, axes = plt.subplots(3, 1, figsize=(12, 10))
        fig.suptitle(title, fontsize=14, fontweight='bold')

        # Orijinal sinyal
        # Matplotlib 3.8+ removed the 'use_line_collection'
            argument from stem
        axes[0].stem(n, np.real(signal), basefmt=' ')
        axes[0].set_xlabel('n (ornek)')
        axes[0].set_ylabel('x[n]')
        axes[0].set_title('Zaman Domeninde Sinyal')
        axes[0].grid(True, alpha=0.3)

        # Genlik spektrumu
        axes[1].plot(self.omega, magnitude, 'b', linewidth
            =2)
        axes[1].set_xlabel('$\omega$ (radyan)')
        axes[1].set_ylabel('|X(e^j$\omega$)|')
        axes[1].set_title('Genlik Spektrumu')
        axes[1].grid(True, alpha=0.3)
        axes[1].set_xlim([-np.pi, np.pi])
        axes[1].set_xticks([-np.pi, -np.pi/2, 0, np.pi/2, np
            .pi])
        axes[1].set_xticklabels(['-π', '-π/2', '0', 'π/2', '
            π'])

        # Faz spektrumu
        axes[2].plot(self.omega, phase, 'r', linewidth=2)
        axes[2].set_xlabel('$\omega$ (radyan)')
        axes[2].set_ylabel('radyan)')
        axes[2].set_title('Faz Spektrumu')
        axes[2].grid(True, alpha=0.3)
```

```
        axes[2].set_xlim([-np.pi, np.pi])
        axes[2].set_xticks([-np.pi, -np.pi/2, 0, np.pi/2, np
            .pi])
        axes[2].set_xticklabels(['-π', '-π/2', '0', 'π/2', '
            π'])
        axes[2].set_ylim([-np.pi, np.pi])
        axes[2].set_yticks([-np.pi, -np.pi/2, 0, np.pi/2, np
            .pi])
        axes[2].set_yticklabels(['-π', '-π/2', '0', 'π/2', '
            π'])

        plt.tight_layout()
        return fig
```

Listing 2: dtftanalyzer.py

```
import numpy as np

class SignalGenerator:

    @staticmethod
    def exponential_decay(a, length=50):
        n = np.arange(0, length)
        return (a ** n) * (n >= 0), n

    @staticmethod
    def complex_exponential(a, omega, length=40):
        n = np.arange(0, length)
        return (a ** n) * np.exp(1j * omega * n) * (n >= 0),
             n

    @staticmethod
    def rectangular_window(width=5):
        n = np.arange(-width*2, width*2+1)
        return (np.abs(n) <= width).astype(float), n

    @staticmethod
    def sinusoidal(freq, length=50, fs=1.0):
        n = np.arange(0, length)
        return np.sin(2 * np.pi * freq * n / fs), n
```

Listing 3: signalgenerator.py